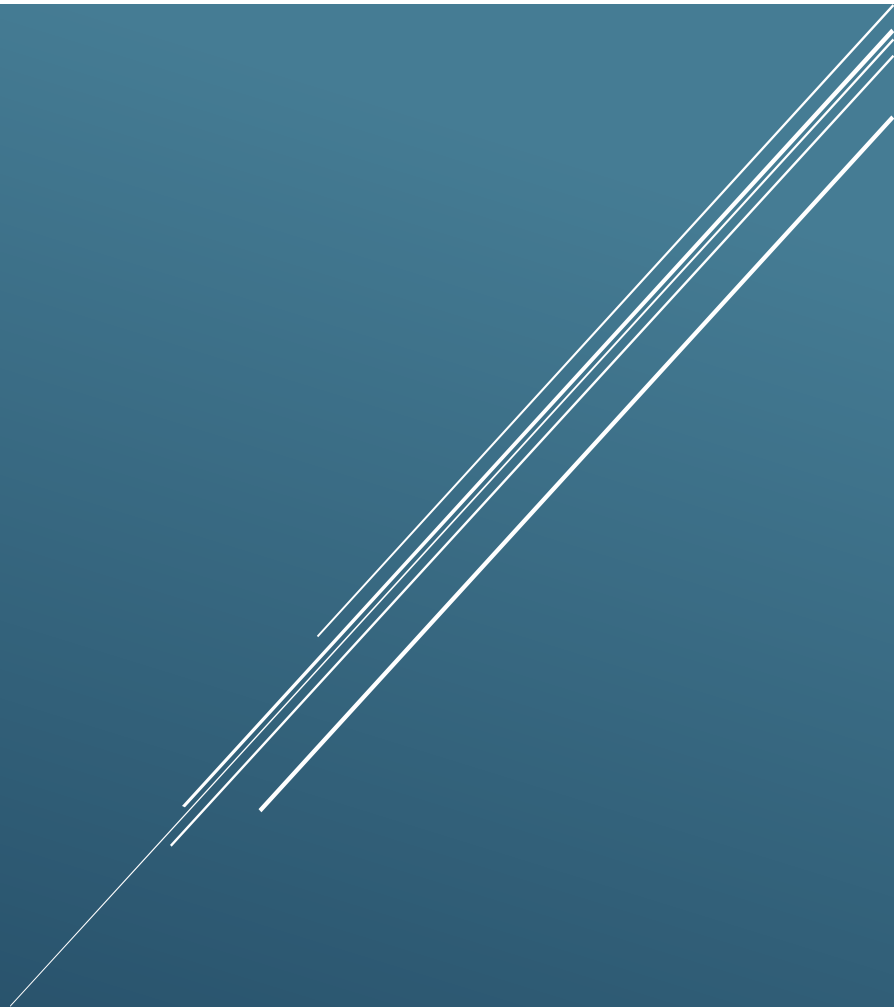




TURBULENCE METHODS

Microstructure profilers, ADCPs, Wire-Walker,
Moored thermistors, Eddy correlation

Table of Contents

1.	Microstructure profilers	2
1.1.	Data	2
1.2.	Scripts structure, overview of the analysis	2
1.3.	Metadata information, key parameters	2
1.4.	STEP 1 – Configuration file update	4
1.5.	STEP 2 – Profile extraction	4
1.6.	STEP 3 – FP07 calibration	4
1.7.	STEP 4 – Odas shear analysis (quick_look)	4
1.8.	STEP 5 – Salinity & density	4
1.9.	STEP 6 – Pressure correction	4
1.10.	STEP 7 – Thorpe displacements	4
1.11.	STEP 8 – Bins creation	4
1.12.	STEP 9 – Filtering & despiking.....	4
1.13.	STEP 10 – Diffusivity and viscosity.....	4
1.14.	STEP 11 – Acceleration, vibration and inclination flagging	4
1.15.	STEP 12 – Shear spectral analysis	4
1.16.	STEP 13 – FP07 spectral analysis	4
1.17.	STEP 14 – Spectral analysis flagging.....	9
1.18.	STEP 15 – Eddy diffusivity and Ozmidov length scale	9
2.	ADCP	10
3.	Wire-walker.....	10
4.	Moored thermistor.....	10
5.	Eddy correlation	10

1. Microstructure profilers

1.1. Data

Data collection (sensors, methods) & format

1.2. Scripts structure, overview of the analysis

1.3. Metadata information, key parameters

Parameters specified in the structure "param.info" in the function "load_parameters":

Parameter	Unit	Type	Meaning	Default value
system	-	String	Name of the lake	'default'
pmin	dbar	Double/Float	Minimum depth to start the profiles in the first profile detection algorithm	0
pmax	dbar	Double/Float	Maximum depth of the binned profiles	30
dp	m	Double/Float	Bin overlap	0.5
dpD	m	Double/Float	Bin size	1
prof_dir	-	String ('up' or 'down')	Profiling direction	'down'
mindur_detect	s	Double/Float	Minimum duration of the profiles to detect	30
minvel_detect	m/s	Double/Float	Minimum velocity of the profiles to detect	0.1
minKT			?	1
fAA	Hz	Double/Float	Maximum frequency for calculations fn (90% of anti-aliasing filter f_filter. E.g., if f_filtter=98 Hz, fn=0.9*98=88.2 Hz)	90
Tspec	-	String ('K'=Kraichnan or 'B'=Batchelor)	Reference temperature spectrum to fit to the observations	'K'
q	-	Double/Float	Turbulent parameter (constant) in the theoretical temperature spectrum	5.26
num_fft	-	Integer	?	3
int_range	-	String ('L'=Luketina and Imberger, 2001 or 'S'=Steinbuck, 2009)	Integration range	'L'

time_corr	-	String ('RSI', 'KOC'= Kocsis, 'NAS'=Nash et al., 1999)	FP07 time response correction method determining the parameter “tau”	‘NAS’
time_res	s	Double/Float	Nominal FP07 response time tau_0 (only used with ‘KOC’ and ‘NAS’ methods)	0.0058
Nasmyth_spec	-	String (‘ODAS’ or ‘EPFL’)	Method for Nasmyth spectral fitting ?	‘EPFL’
noise_corr	-	String (none or ‘Goodman’)	Method for noise correction ?	‘Goodman’
npoles	-	String (single' or 'double')	Type of FP07 time response correction	‘single’
kmax_factor				1/1.66
despike_sh		1*3 array of double/floats	Parameters to remove spikes in shear data (ODAS function “despike”)	[8 0.5 0.04]
despike_A		1*3 array of double/floats	Parameters to remove spikes in accelerometer data (ODAS function “despike”)	[8 0.5000 0.0400]
peak_rem_T1, peak_rem_T2				[0,0]
noisep_T1, noisep_T2				[-10.08+0.25,-0.97,param.info.fAA]
ksfact	m ⁻¹	Double/Float	Upper bound of the inertial-convective subrange, as in Steinbuck et al. (2009)	0.04
Snfact	-	Double/Float	Maximum acceptable signal to noise ratio, as in Goto et al. (2016)	1.55

1.4. STEP 1 – Configuration file update

1.5. STEP 2 – Profile extraction

1.6. STEP 3 – FP07 calibration

1.7. STEP 4 – Odas shear analysis (quick_look)

1.8. STEP 5 – Salinity & density

1.9. STEP 6 – Pressure correction

1.10. STEP 7 – Thorpe displacements

1.11. STEP 8 – Bins creation

Bin-averages

1.12. STEP 9 – Filtering & despiking

1.13. STEP 10 – Diffusivity and viscosity

1.14. STEP 11 – Acceleration, vibration and inclination flagging

1.15. STEP 12 – Shear spectral analysis

Two theoretical spectra: Panchev & Kesich or Nasmyth (see Sanchez et al., 2011)

1.16. STEP 13 – FP07 spectral analysis (function gradT_dis_spec.m)

Two theoretical spectra: Batchelor or Kraichnan

1.16.1. Temperature gradient detrending

1.16.2. Power spectral density computation

The power spectral density of the fluctuations (with respect to the bin average) of the detrended temperature gradient $x = \text{gradT}$ [$^{\circ}\text{C}/\text{m}$] is computed with `csd_odas()` using overlapping segments of length “sl”.

```
% This function is provided by RSI in the ODAS libraries.  
% Alternatively, pwelch can be used obtaining comparable results.  
[PSDT,fr] = csd_odas(x-nanmean(x),x-nanmean(x),sL,scalar_info.fs,[],sOV,'linear');  
% [PSDT,fr] = pwelch(x-nanmean(x),hann(sL),sOV,sL,scalar_info.fs,'onesided');
```

Where PSDT [$^{\circ}\text{C}^2.\text{s}.\text{m}^{-2}$] is the auto spectrum of x and fr [s^{-1}] are the corresponding frequencies ranging from 0 to the Nyquist frequency (half of the sampling frequency $f_s/2$).

1.16.3. Conversion from time to space

The frequency spectrum is converting to the space domain (wavenumbers) using Taylor’s frozen hypothesis that states that temporal changes in temperature measured at one location are spatial changes in temperature that are advected by turbulent eddies at the “flow” velocity W

(sinking speed of the profiler here). As a result, temporal frequencies f are directly related to wavenumbers $k = f/W$.

```
% convert the frequency spectrum into the corresponding wavenumber spectrum
(Taylor's frozen turbulence hypothesis)
PSDT=PSDT*W;
k = fr/W;
PSD=PSDT;
```

1.16.4. Time response correction

Correct the spectrum based on the FP07 time response. Three methods can be used to determine the parameter τ that defines H from either single pole or double poles correction.

```
% Time-response correction
if strcmp(time_corr,'KOC') % as in Kocsis et al. (1999)
    tau = tau_0*W.^-0.5;
elseif strcmp(time_corr,'RSI') % as in Vachon and Lueck (1984)
    F0 = 25*sqrt(W);
    tau = (2*pi()*F0/sqrt(sqrt(2)-1))^( -1);
elseif strcmp(time_corr,'NAS') % as in Nash et al. (1999)
    tau = tau_0*W.^-0.12;
elseif strcmp(time_corr,'SOM') % as in Sommer et al. (2013)
    tau = 0.010;
else
    error('Error: time_corr can be RSI, KOC or NAS')
end

% Single or double correction
if strcmpi(npoles,'single')
    H = 1./(1 + (2*pi()*tau*fr).^2);
    H_lim=0.1;
elseif strcmpi(npoles,'double')
    H = 1./(1 + (2*pi()*tau*fr).^2).^2;
    H_lim=0.1;
end
PSD_raw=PSD; % Raw spectrum
PSD = PSD./H; % Corrected spectrum
```

1.16.5. Noise spectrum

The noise spectrum is computed with the ODAS function `gradT_noise_odas()` or with the function `FP07noise()`.

```
% Noise function (according to the function provided by RSI in the ODAS
libraries)
noise_info=gradT_noise_odas;
noise_info.gamma_RSI = 1;
noise_info.E_n = 3e-9;
Sn = gradT_noise_odas(T_dT, T_string, W, fr, setupstr,noise_info); % Noise
spectrum [(K/m)^2/Hz].
Sn = Sn*W;
Sn = Sn./H;
Sn(1) = 0;

% Alternatively, the noise can be computed as follows
% noisep = sensor dependent parameters. E.g. [-10.24,-0.89,fAA]
```

```

% where fAA = frequency of the anti-aliasing filter
% Sn = FP07noise(noisep,fr);
% Sn = Sn*W;
% Sn = Sn.*(2*pi()*k).^2;
% Sn = Sn./H;
% Sn(1) = 0;

```

1.16.6. Integration range

The fitting is performed in the noise-free range of the spectrum.

Determines the initial range limits:

```

iknM0 = find(PSD<Snfact*Sn | H<H_lim | k==max(k),1,'first'); % Upper wavenumber
limit of the fitting range

kS = ksfact*Pr^(-0.5)*kB_S; % Lower wavenumber limit according to Steinbuck et al.
(2009)
k11 = max([kS,k1]);
ik11 = find(k>=k11,1,'first');
ikcor = ik11:iknM0;
Xiv = 6*D*sum( (PSD(ikcor(2:end))+PSD(ikcor(1:end-1))).*(k(ikcor(2:end))-
k(ikcor(1:end-1))) )/2;
Xin = 6*D*sum( (Sn(ikcor(2:end))+Sn(ikcor(1:end-1))).*(k(ikcor(2:end))-
k(ikcor(1:end-1))) )/2;

%% A signal-to-noise ratio larger than 1.3 was required in the integration range
if Xiv>1.3*Xin
    cont = true;
    Xiv = Xiv -Xin;
else
    Xiv = 0;
    Xi_T = 0;
end

```

1.16.7. Fitting

Fitting is performed for wavenumbers ranging from $\max([kS, kuK])$ to upper value defined from signal-to-noise ratio. The range limits are adapted during the fitting procedure, by doing three “searches”. The function `cost_T_fit()` is used for fitting the observed PSD to the theoretical spectrum.

```

%% First search
for i = 1:length(ktest)
    kS = k1; % The Steinbuck et al. (2009) lower limit is not considered
during the first round
    kL = max([kS,k1]);
    ikL = find(k>=kL,1,'first');

```

```

        ikfit = ikL:iknM0;
        [cost(i), Xif0(i),Steo] = cost_T_fit(k, PSD, ktest(i), ikfit, Tdis, q,
dof, Sn,D,Snfact);
    end

    LKHtest = - cost;
    ML = max(LKHtest);
    iML = find(LKHtest == ML);
    kB00 = ktest(iML);
    BATf0 = Tspec(Tdis,Xif0(iML),kB00,k,D,q);
    kP=kB00/sqrt(6*q); % k at the peak

    %% Look for the lower integration limit kL
    if strcmp(int_range,'S') % If only Steinbuck et al. (2009) condition
        kLuk=k1; islope=2;
    elseif strcmp(int_range,'L') % If only Steinbuck et al. (2009) + Luketina and
Imberger (2001) conditions
        % Find the intersection between observed and theoretical spectra
        diffS = (PSD-BATf0)./abs(PSD-BATf0);
        ikL = find(diffS<0,1,'first');
        if ~isempty(ikL) && abs(PSD(ikL)-BATf0(ikL))>abs(PSD(ikL-1)-BATf0(ikL-1))
            ikL=ikL-1;
        end
        % This was not mentioned in Luketina, it is a double check on
        % the existence of an actual finestructure pattern.
        % Calculate the slope of the first part of the spectrum using an
increasing window
        % Note: k(1) and PSD(1) = 0 --> the window starts from the 2nd point.
        % The iteration starts from the 4th point to have at least 3 points to
calculate the slope
        ikP = find(k<=kP,1,'last');
        slope=NaN(1,iknM0);
        for i=4:iknM0
            X=log10(k(2:i)); Y=log10(PSD(2:i));
            X=[ones(length(X),1) (X)];
            b=X\Y; % \ computes the least square fit
            slope(i)=b(2);
        end
        if ~isempty(ikL)
            [vslope,islope]=nanmin(slope(1:min(ikP,ikL))); % find the minimum of
the slope
            if vslope>0 || nanmean(slope(1:islope))>0 % if the minimum slope is >
0, then
                islope=2;
            end
        else
            islope=2;
        end
        kLuk = min([k(islope),k(ikL),kP]);
    end

    %% Second search
    cost = nan(size(ktest));
    Xif0 = nan(size(ktest));
    dk = ktest(2)-ktest(1);
    for i = 1:length(ktest)
        kS = ksfact*Pr^(-0.5)*kB00; % Note: kS is based on the previous best
        kL = max([kS,kLuk]); % Note, if int_range=='L' here we consider
the Luketina and Imberger (2001) condition
    end

```

```

        ikL = find(k>=kL,1,'first');
        ikfit = ikL:iknM0;
        [cost(i), Xif0(i)] = cost_T_fit(k, PSD, ktest(i), ikfit, Tdis, q, dof,
Sn,D,Snfact);
    end
    LKHtest = - cost;
    ML = max(LKHtest);
    iML = find(LKHtest == ML);
    kB00 = ktest(iML);
    BATf0 = Tspec(Tdis,Xif0(iML),kB00,k,D,q);
    kP=kB00/sqrt(6*q); % k at the peak

    %% Update the lower integration limit
    if strcmp(int_range,'S') % If only Steinbuck et al. (2009) condition
        kLuk=k1;
    elseif strcmp(int_range,'L') % If only Steinbuck et al. (2009) + Luketina and
Imberger (2001) conditions
        diffS = (PSD-BATf0)./abs(PSD-BATf0);
        ikL = find(diffS<0,1,'first');
        ikP = find(k<=kP,1,'last');
        if ~isempty(ikL) && abs(PSD(ikL)-BATf0(ikL))>abs(PSD(ikL-1)-BATf0(ikL-1))
            ikL=ikL-1;
        end
        if ~isempty(ikL)
            [vslope,islope]=nanmin(slope(1:min(ikP,ikL))); % find the minimum of
the slope
            if vslope>0 || nanmean(slope(1:islope))>0 % if the minimum slope is >
0, then
                islope=2;
            end
            else
                islope=2;
            end
            kLuk = min([k(islope),k(ikL),kP]);
        end
        ikL = find(k>=kLuk,1,'first');

    %% Third, refined search
    MLp = -cost_T_fit(k, PSD, kB00+dk, ikfit, Tdis, q, dof, Sn,D,Snfact);
    MLM = -cost_T_fit(k, PSD, kB00-dk, ikfit, Tdis, q, dof, Sn,D,Snfact);
    deltak0=abs((2*dk)/sqrt(2*ML-MLm-MLp));
    deltak = max([deltak0,dk]);

    kmin2 = max([kB00-deltak,k(ikL+1)]);
    kmax2 = kB00+deltak;
    ktest = linspace(kmin2,kmax2,40);
    clear cost Xif0
    cost = nan(size(ktest));
    Xif0 = nan(size(ktest));
    for i = 1:length(ktest)
        kS = ksfact*Pr^(-0.5)*kB00; % Note: kS is based on the previous best
        kL = max([kS,kLuk]); % Note, if int_range=='L' here we consider
the Luketina and Imberger (2001) condition
        ikL = find(k>=kL,1,'first');
        ikfit = ikL:iknM0;
        [cost(i), Xif0(i)] = cost_T_fit(k, PSD, ktest(i), ikfit, Tdis, q, dof,
Sn,D,Snfact);
    end

```

```

LKHtest = - cost;
MLKH = max(LKHtest);
iML = find(LKHtest == MLKH);
Xi_T = Xif0(iML);
kB = ktest(iML);
kP=kB/sqrt(6*q); % k peak
BATf = Tspec(Tdis,Xi_T,kB,k,D,q);

```

1.16.8. Specific wavenumbers

- Kolmogorov wavenumber $k_K = \left(\frac{\nu^3}{\varepsilon}\right)^{\frac{1}{4}}$ (not calculated ?): lowest limit of the wavenumber range for which viscous dissipation dominates (based on kinematic viscosity) = where spectral fitting can be done.
- Batchelor wavenumber $k_B = \left(\frac{\nu D_T^2}{\varepsilon}\right)^{\frac{1}{4}} > k_K$: lowest limit of the wavenumber range for which molecular thermal diffusion dominates over turbulence.
- k_S
- k_L
- $k_n = f_n/W$: maximal wavenumber for calculations, based on the anti-aliasing filter frequency
- Wavenumber above which noise dominates (i.e., SNR becomes lower than threshold SNR_{min}): k_{noise} such that $PSD(k_{noise}) \leq SNR_{min} * S_n(k_{noise})$ with $SNR_{min} = 1.55$ (Goto et al., 2018)
- Wavenumber above which time response correction factors is below threshold value $H < H_{lim}$
- $k_P = \frac{k_{B00}}{\sqrt{6q}}$: wavenumber corresponding to the peak of the fitted theoretical spectrum, where k_{B00} is the Batchelor wavenumber that gives the maximum likelihood

1.17. STEP 14 – Spectral analysis flagging (quality metrics)

1.17.1. Mean Absolute Deviation (MAD)

Criterion to keep the data: $MAD < 2 * MAD_c$

1.17.2. Likelihood Ratio (LR)

Ratio between the likelihood from theoretical spectrum and likelihood power-law fitting: should be large.

Criterion to keep the data: $LR < 2$ (log10 space)

1.17.3. Signal-to-Noise criterion?

1.17.4. Confidence Interval?

1.18. STEP 15 – Eddy diffusivity and Ozmidov length scale

2. ADCP

3. Wire-walker

4. Moored thermistor

5. Eddy correlation